

Reviewer Pack — Schur-Weyl Decomposition Rank Lower Bounds Disjointness Comm...

Entry c2c1aaf5edd5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 19:28:01 UTC

Schur-Weyl Decomposition Rank Lower Bounds Disjointness Communication Complexity

Entry ID: c2c1aaf5edd5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 19:28:01 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality **Field B** (complexity object): Disjointness communication complexity

Statement:

For any bipartite graph $G = (A \cup B, E)$ with $|A| = |B| = n$, let $\lambda(G)$ denote the minimal number of complete bipartite subgraphs needed to cover E . The Schur-Weyl decomposition rank $R(G)$ of the adjacency matrix equals $\lambda(G)$. For all G with $\lambda(G) \geq 2$, $R(G) \geq \Omega(n^{1/2})$

Rationale (proposer's reasoning):

Schur-Weyl duality provides a decomposition of tensor products into irreducible representations, which can quantify symmetry in bipartite graphs. This decomposition rank captures structural properties of edge covers, directly linking to disjointness protocols' inherent communication requirements via the covering number's combinatorial lower bounds.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 89efa5cb53b2a9f2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_bipartite_graph(n):
    A = list(range(n))
    B = list(range(n, 2*n))
    E = set()
    for _ in range(int(n * (n - 1) / 4)):
        u = random.choice(A)
        v = random.choice(B)
        if (u, v) not in E and (v, u) not in E:
            E.add((u, v))
    return A, B, E

def max_matching(G):
    n = len(G[0])
    matching = [-1] * n
    visited = [False] * n

    def dfs(u):
        for v in G[u]:
            if not visited[v]:
                visited[v] = True
                if matching[v] == -1 or dfs(matching[v]):
                    matching[v] = u
                    return True
        return False

    for u in range(n):
        visited = [False] * n
        dfs(u)

    return sum(1 for x in matching if x != -1)

def schur_weyl_rank(G):
    A, B, E = G
    n = len(A)
```

```

M = [[0] * (n + n) for _ in range(n + n)]

for u, v in E:
    M[u][v], M[v][u] = 1, 1

def gaussian_elimination(M):
    m, n = len(M), len(M[0])
    rank = 0
    for j in range(n):
        i_max = -1
        for i in range(rank, m):
            if M[i][j]:
                i_max = i
                break
        if i_max == -1:
            continue
        M[rank], M[i_max] = M[i_max], M[rank]
        rank += 1
        for i in range(rank, m):
            factor = M[i][j] / M[rank-1][j]
            for k in range(n):
                M[i][k] -= factor * M[rank-1][k]
    return rank

return gaussian_elimination(M)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    G = generate_bipartite_graph(n)
    lambda_G = max_matching(G)
    R_G = schur_weyl_rank(G)

    metric_name = "R(G)"
    metric_value = R_G
    instances_tested = 1
    conjecture_holds = R_G >= math.sqrt(n) / 2
    counterexample = "" if conjecture_holds else f"R(G)={R_G} < {math.sqrt(n)/2}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_646169d4.py", line 104, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_646169d4.py", line 81, in run_trial
    lambda_G = max_matching(G)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_646169d4.py", line 44, in max_matching
    dfs(u)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_646169d4.py", line 35, in dfs
    if not visited[v]:
        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError, preventing validation of conjecture. The error suggests a bug in graph processing code that needs fixing before results can be assessed. | next: Fix the index out of range error in max_matching function by adding bounds checking for node indices

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	30045
2	preregistration	ollama_remote	qwen3:8b	0	0	24146
3	novelty	ollama_remote	qwen3:8b	0	0	20597
4	novelty	ollama_remote	qwen3:8b	0	0	18210
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16512
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10631
7	judge	ollama_remote	qwen3:8b	0	0	17165

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137306 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/c2c1aaf5edd5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c2c1aaf5edd5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c2c1aaf5edd5.tar.gz` (if generated)